Lec-19

02/05/26 9:47 AM

Given a string `s`, partition `s` such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of* `s`.

Example 1:

```
Input: s = "aab"
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"
Output: [["a"]]
```

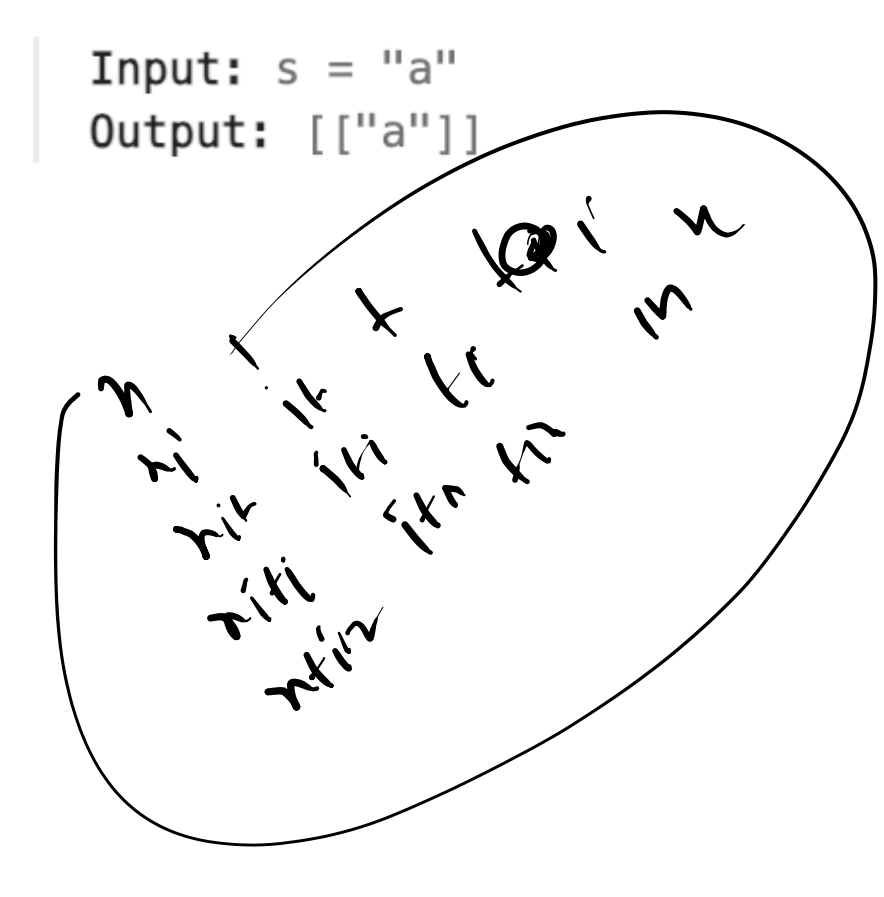
aab

a|a|b

aa|b

ni|t|i|n

nitin


n i t i n
ni it ti in
nit iti tin
niti itin
nitin

nitin [[n,i,t,i,n]

[n,iti,n]

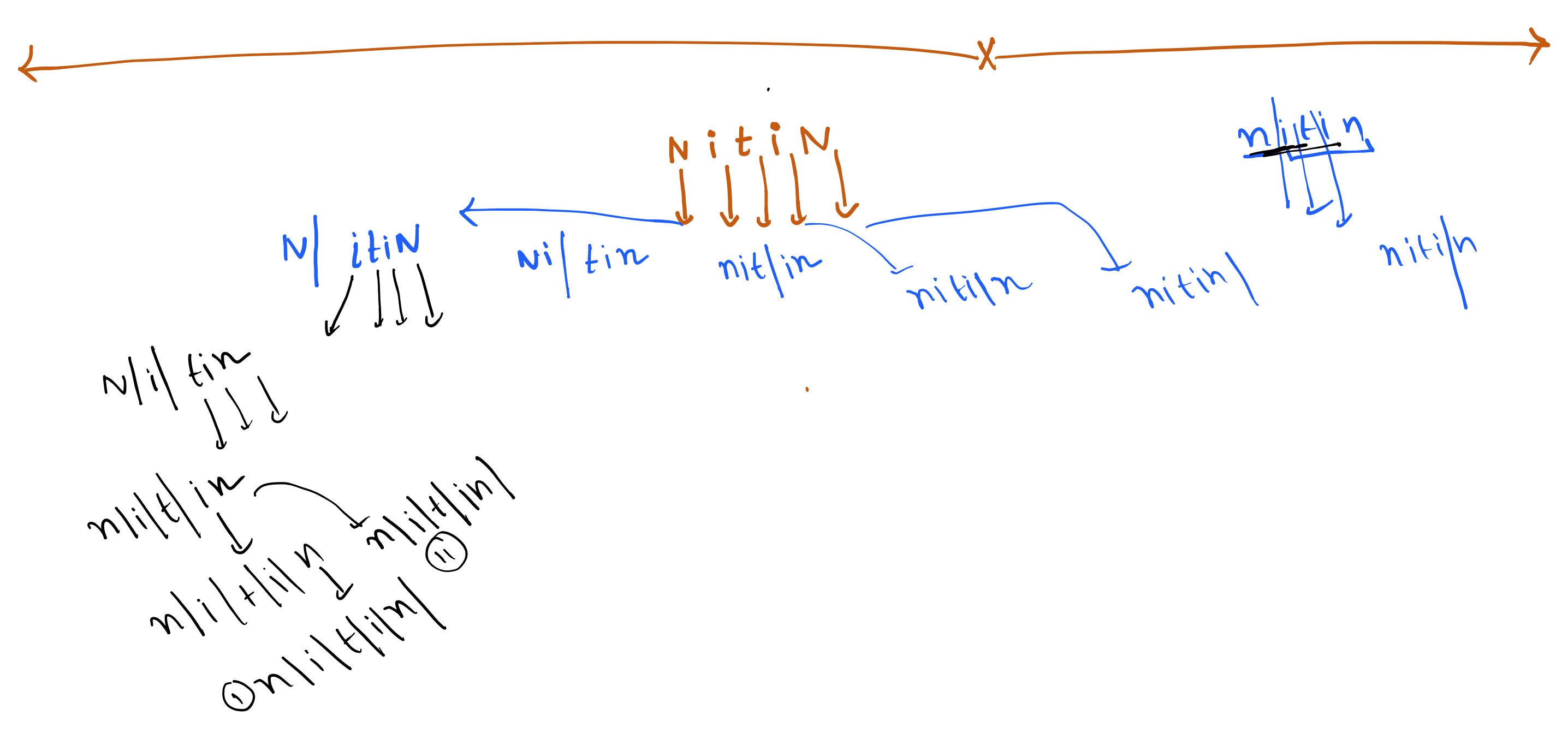
[nitin]]

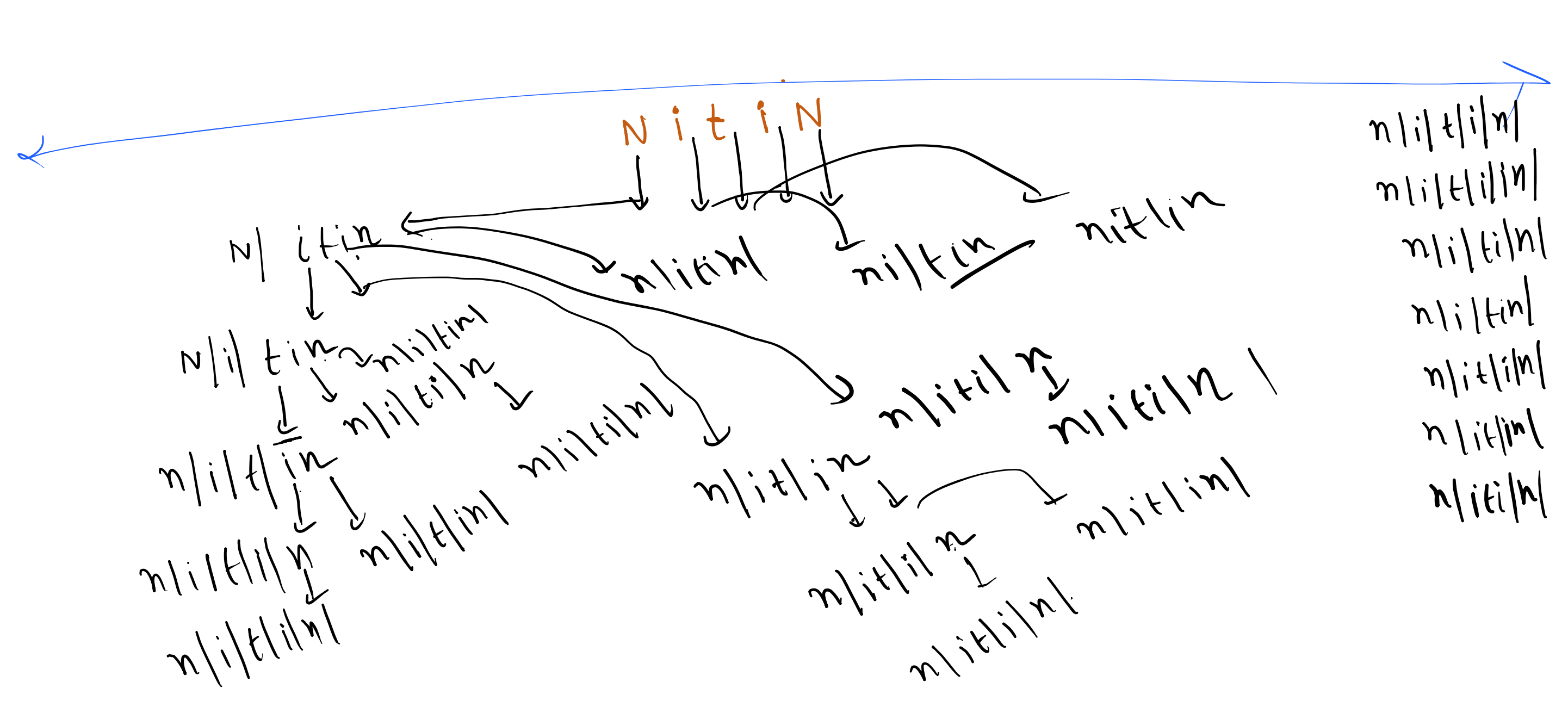
n|i|t|i|n

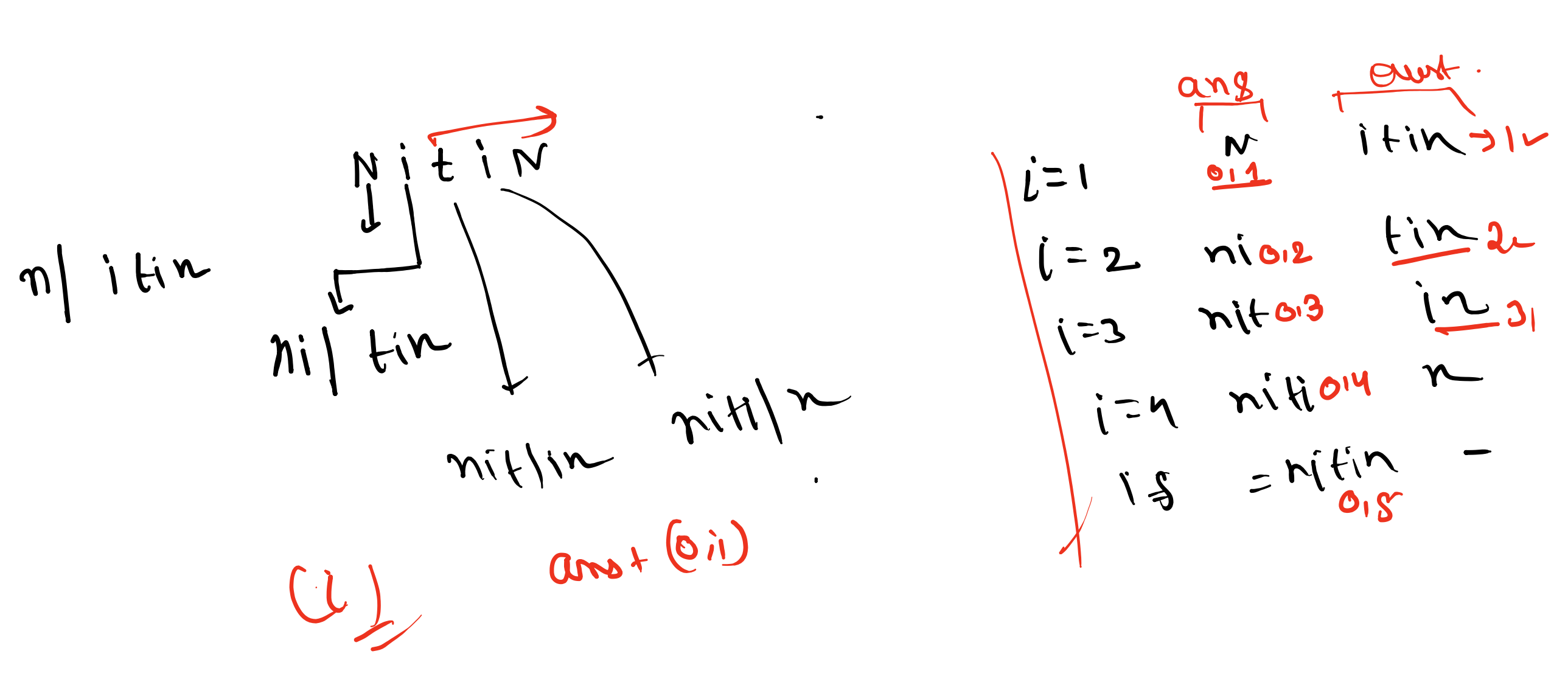
n|iti|n

nitin|

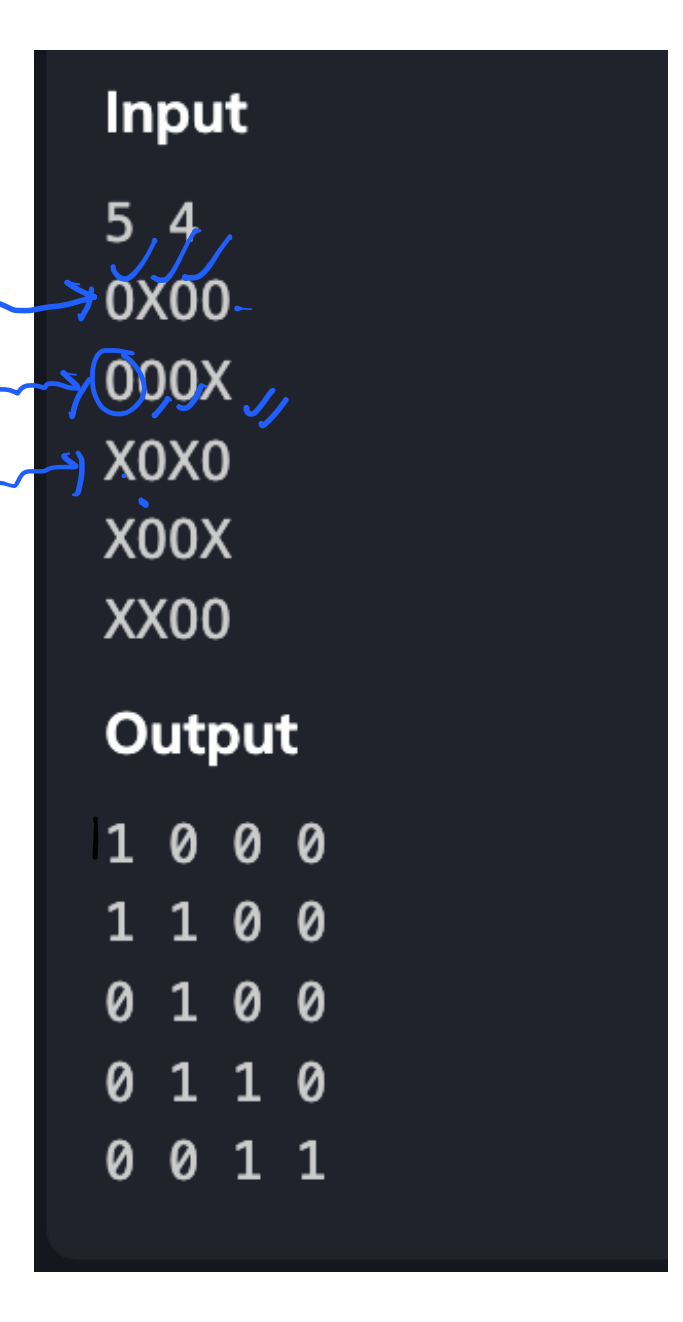
```
n|i|t|i|n|
n|i|t|in|
n|i|ti|n|
n|i|tin|
n|it|i|n|
n|it|in|
n|iti|n|
n|itin|
ni|t|i|n|
ni|t|in|
ni|ti|n|
ni|tin|
nit|i|n|
nit|in|
niti|n|
nitin|
```


X
NitiN
N|itiN
Ni|tin
nit|in
niti|n
nitin|
n|iti|n
N|i|tin
n|i|t|in
n|i|t|i|n
n|i|t|i|n|
n|i|ti|n|

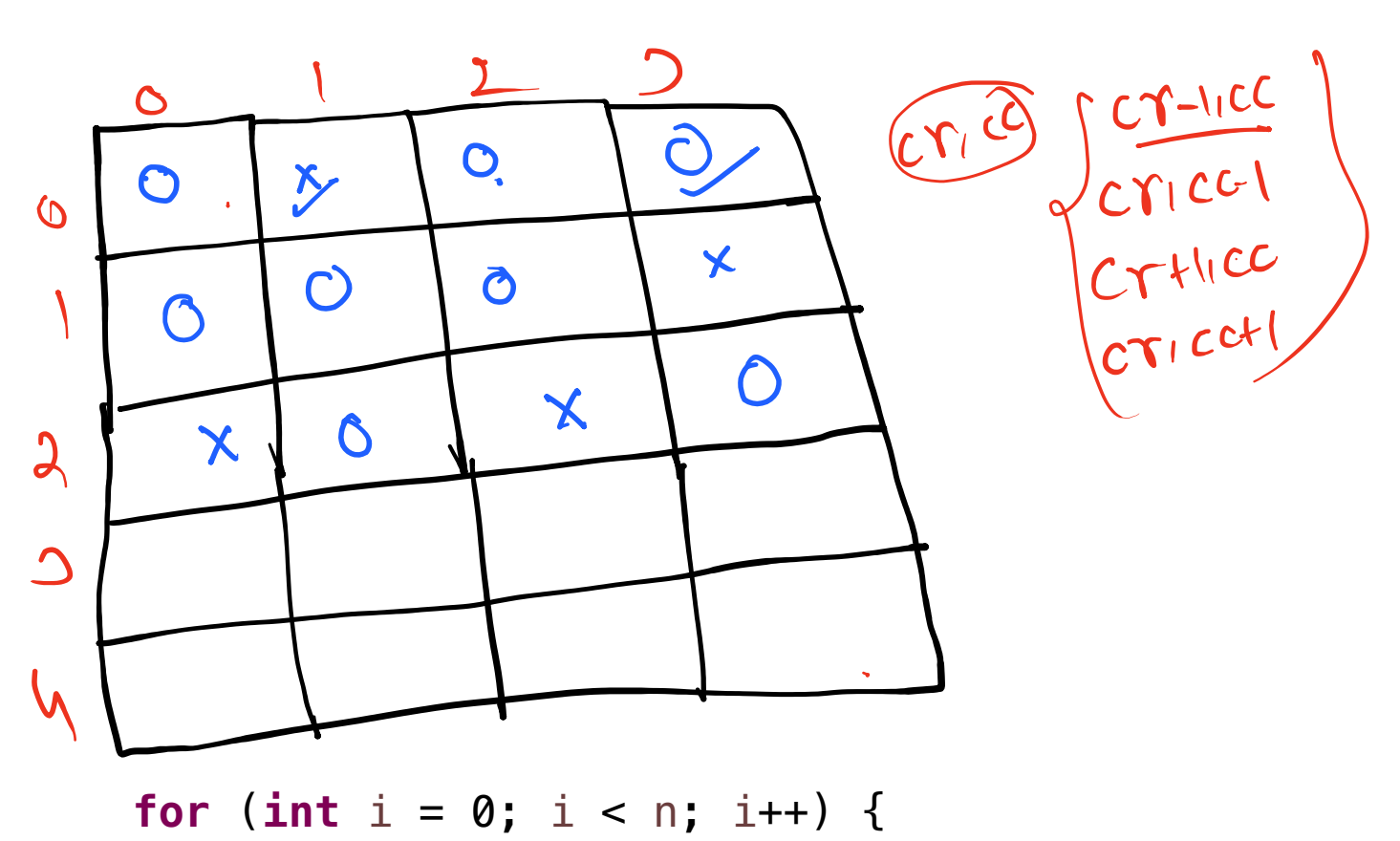

NitiN
N|itin
n|itin|
ni|tin
nit|in
N|i|tin
n|itin|
n|i|ti|n
n|iti|n
niti|n|
n|i|t|in
n|i|ti|n|
n|it|in
n|i|t|i|n
n|i|t|in|
n|it|i|n|
n|it|i|n|
n|i|t|i|n|
n|i|t|i|n|
n|i|ti|n|
n|i|tin|
n|it|i|n|
n|it|in|
n|iti|n|


NitiN
n| itin
ni| tin
nit|in
niti|n
(i)
ans+(0,i)
ans
curr
i=1
i=2
i=3
i=4
i=5
N 0,1
ni 0,2
nit 0,3
niti 0,4
=nitin 0,5
itin 1
tin 2
in 3
n
-

```
n|i|t|i|n|
n|i|t|in|
n|i|ti|n|
n|i|tin|
n|it|i|n|
n|it|in|
n|iti|n|
n|itin|
ni|t|i|n|
ni|t|in|
ni|ti|n|
ni|tin|
nit|i|n|
nit|in|
niti|n|
nitin|
```

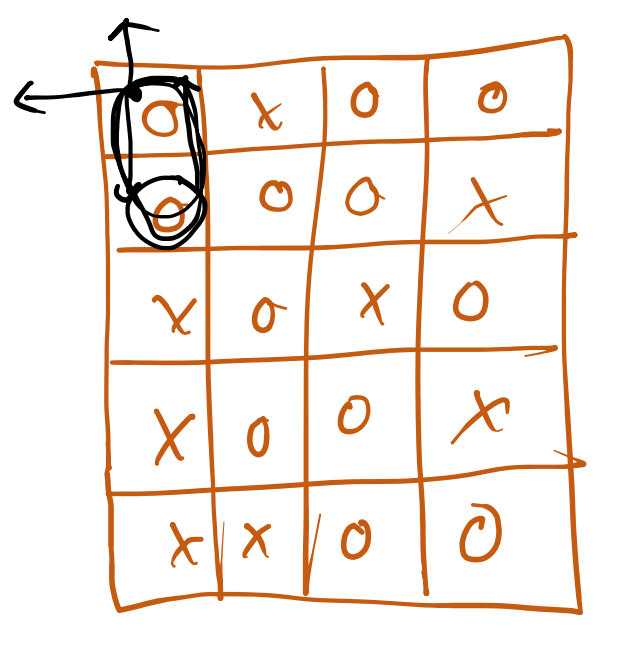


```
Input
5 4
0X00
000X
X0X0
X00X
XX00

Output
1 0 0 0
1 1 0 0
0 1 0 0
0 1 1 0
0 0 1 1
```

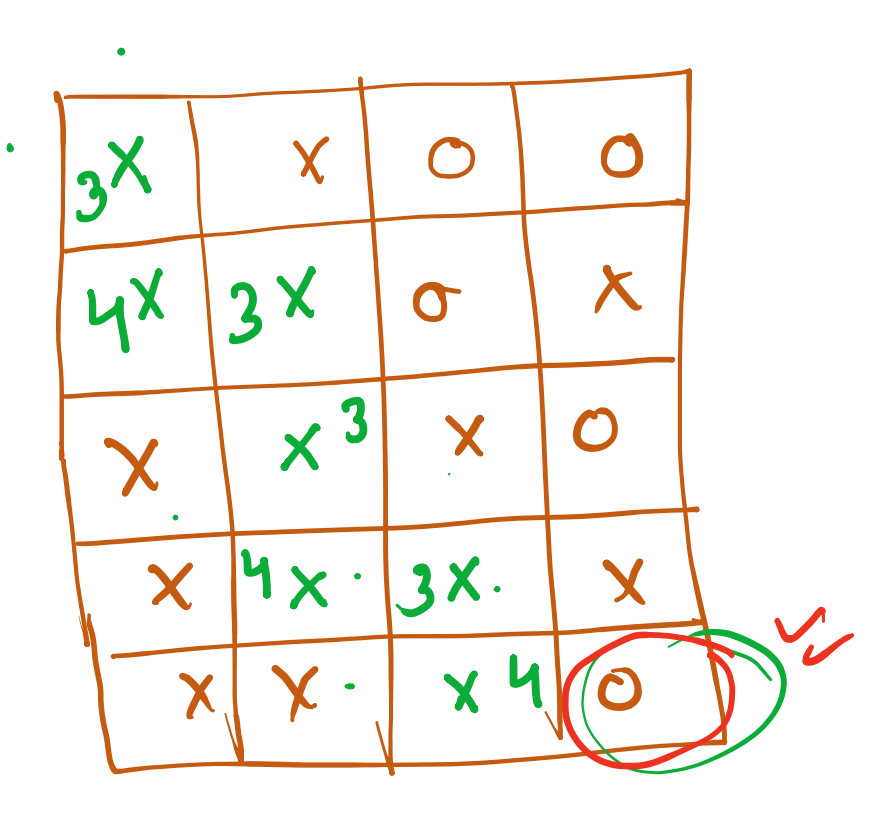

0 1 2 3
0 1 2 3 4
cr,cc
cr-1,cc
cr,cc-1
cr+1,cc
cr,cc+1

```
for (int i = 0; i < n; i++) {
        String s=sc.next();
        for (int j = 0; j < m; j++) {
            maze[i][j]=s.charAt(j);
        }
}
```

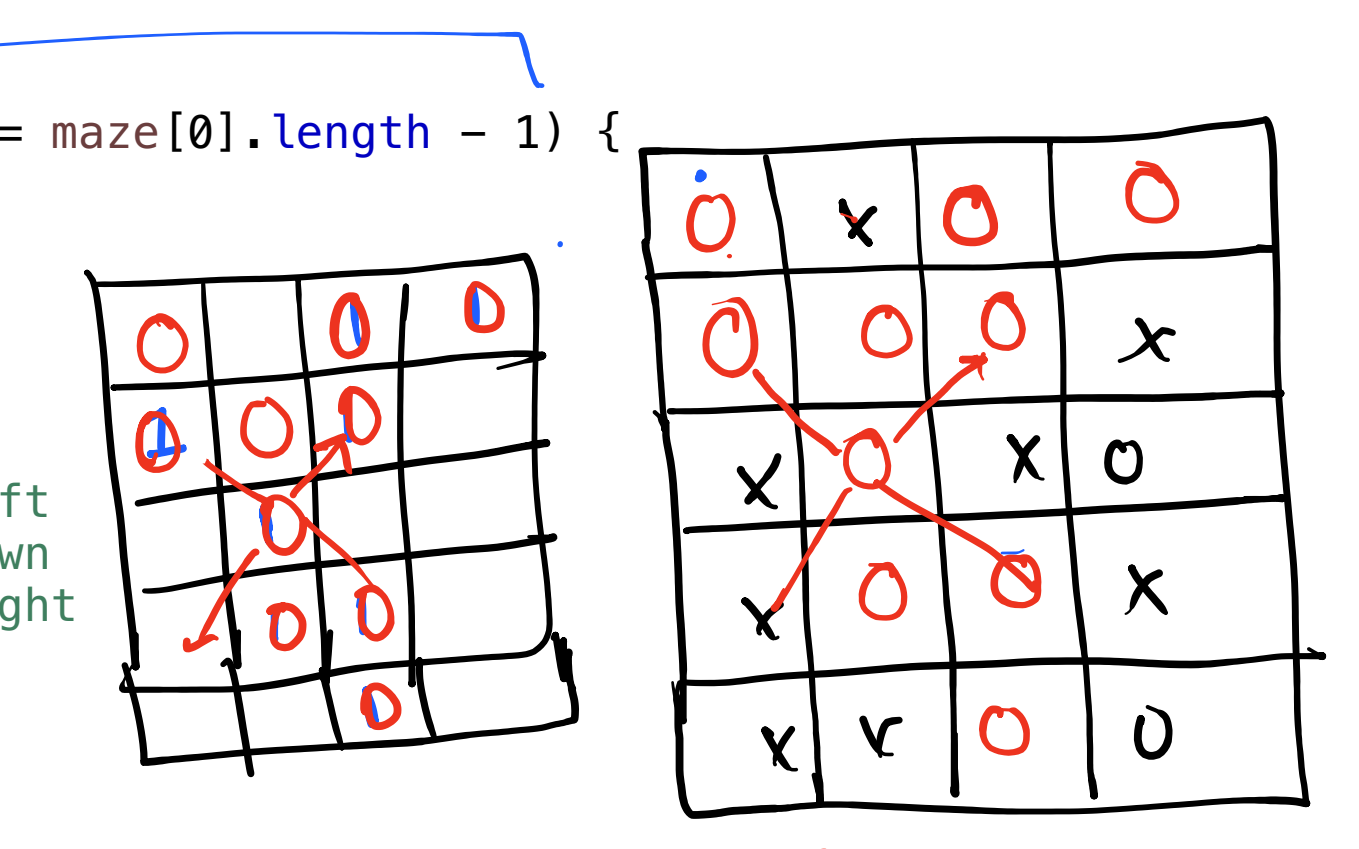
```
public static void Path(char[][] maze, int cr, int cc) {
        // TODO Auto-generated method stub
        if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length) {
            return;
        }
        Path(maze, cr - 1, cc);// UP
        Path(maze, cr, cc - 1);// left
        Path(maze, cr + 1, cc);// down
        Path(maze, cr, cc + 1);// right

    }
```



```
public static void Path(char[][] maze, int cr, int cc) {
        // TODO Auto-generated method stub
        if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length|| maze[cr][cc]=='X') {
            return;
        }
        maze[cr][cc]='X';
        Path(maze, cr - 1, cc);// UP
        Path(maze, cr, cc - 1);// left
        Path(maze, cr + 1, cc);// down
        Path(maze, cr, cc + 1);// right
        maze[cr][cc]='O';

    }
```



```
public static void Path(char[][] maze, int cr, int cc, int[][] ans) {
        // TODO Auto-generated method stub
        if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length || maze[cr][cc] == 'X') {
            return;
        }
        if (cr == maze.length - 1 && cc == maze[0].length - 1) {
            Display(ans);
            return;
        }
        maze[cr][cc] = 'X';
        ans[cr][cc] = 1;
        Path(maze, cr - 1, cc, ans);// UP
        Path(maze, cr, cc - 1, ans);// left
        Path(maze, cr + 1, cc, ans);// down
        Path(maze, cr, cc + 1, ans);// right
        maze[cr][cc] = 'O';
        ans[cr][cc] = 0;

    }
```



R = 2-1, 0 0 1 -1 -1+1 3

C = 2 0 1 -1 1 0 1 -1 +1 3

(i) Count Total PS ✓

(ii) Length longest PS

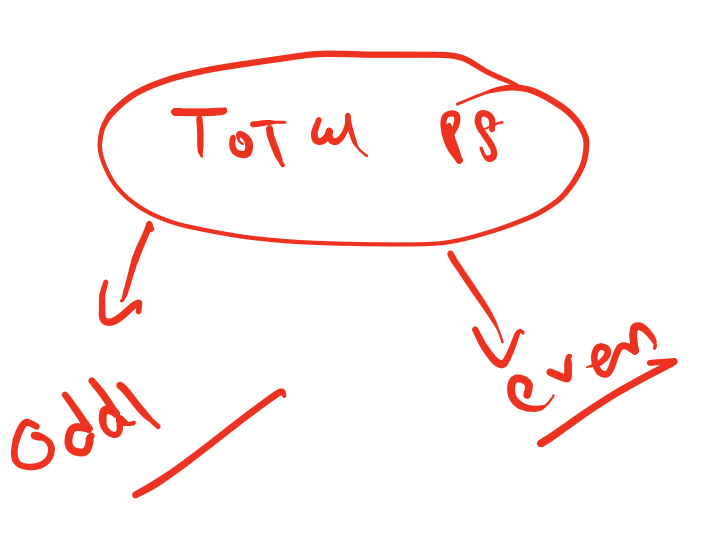
(iii) Print longest PS

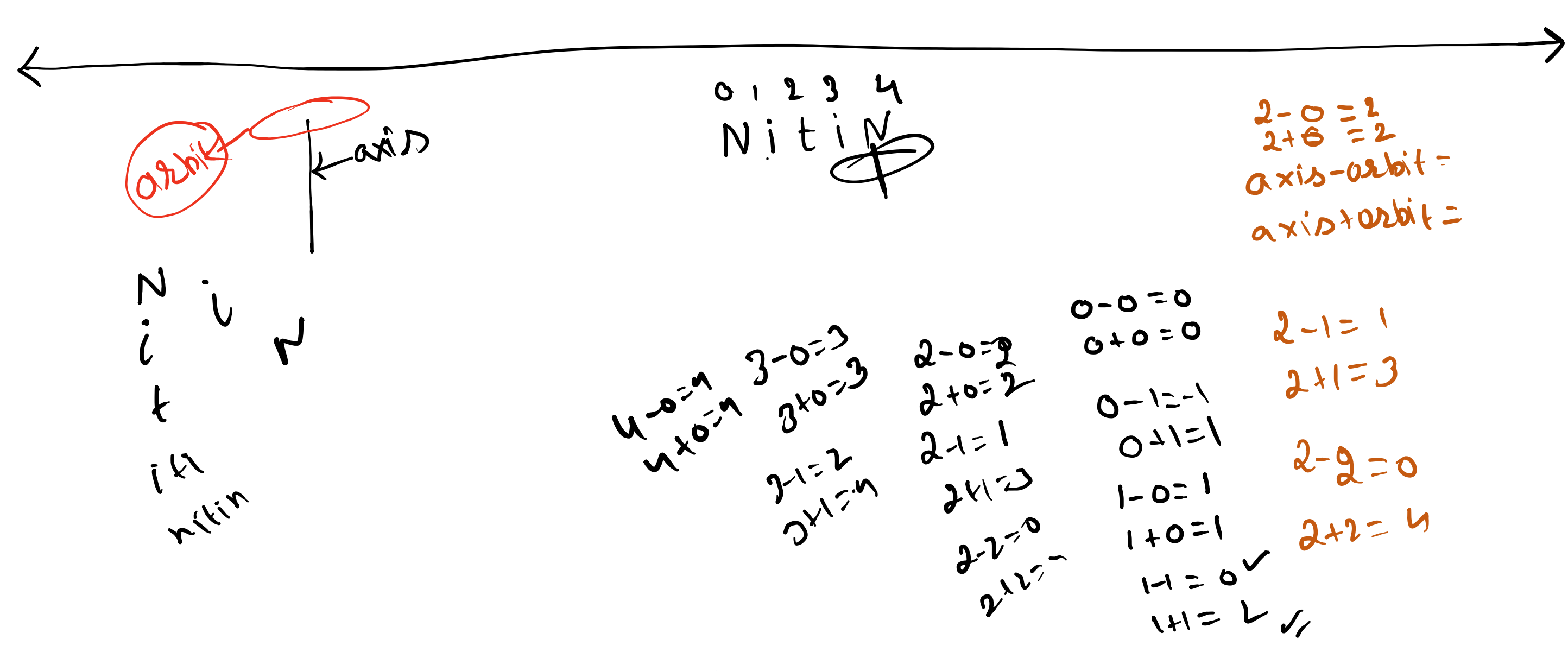
(iiii) Count only odd len PS

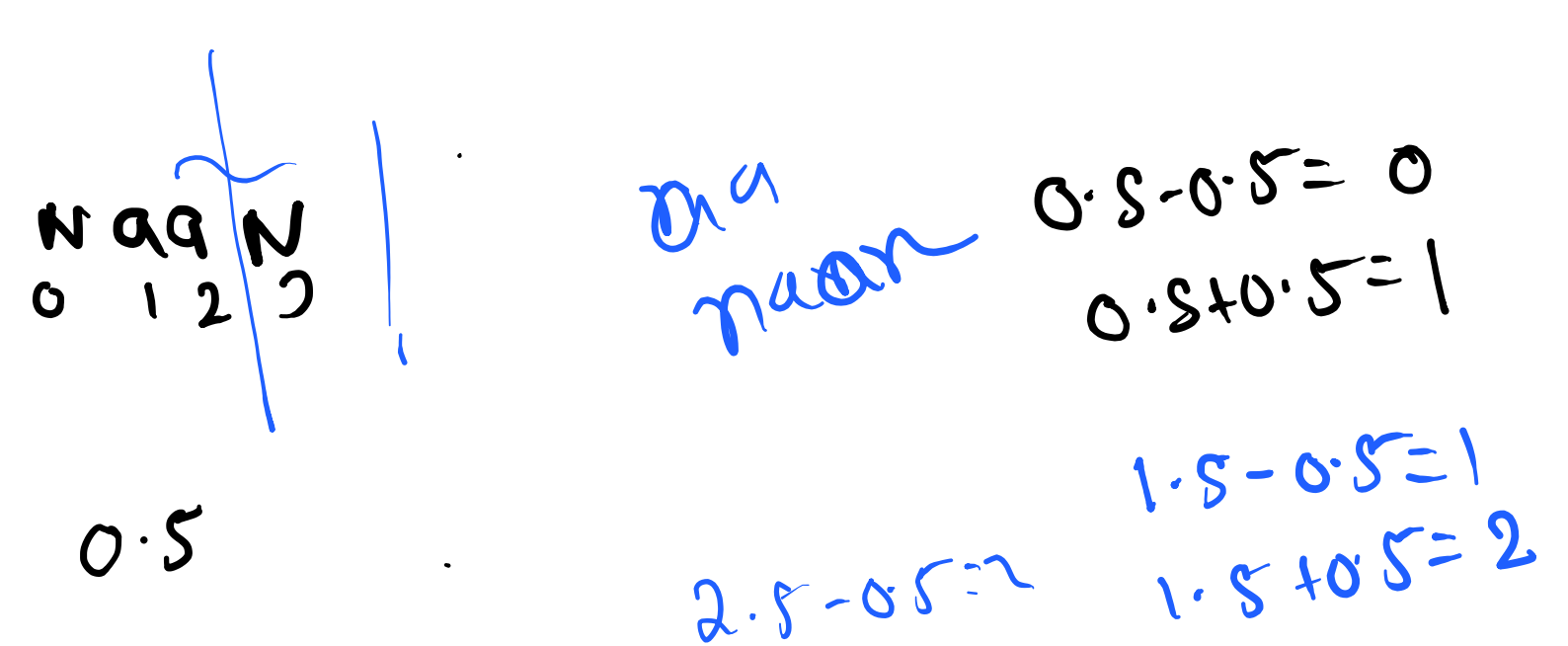
(v) Count only even len PS while even - ( )

odd - ( )

(vi) Length longest PS


Total PS
odd
even


orbit
axis
0 1 2 3 4
NitiN
N
i
t
iti
nitin
i
N
4-0=4
4+0=4
3-0=3
3+0=3
2-1=2
2+1=4
2-0=2
2+0=2
2-1=1
2+1=3
2-2=0
2+2=4
0-0=0
0+0=0
0-1=-1
0+1=1
1-0=1
1+0=1
1-1=0
1+1=2
2-0=2
2+0=2
axis-orbit=
axis+orbit=
2-1=1
2+1=3
2-2=0
2+2=4


N a a N
0 1 2 3
0.5
aa
naan
0.5-0.5=0
0.5+0.5=1
1.5-0.5=1
1.5+0.5=2
1.5-1.5=0
1.5+1.5=3
2.5-0.5=2
2.5+0.5=3

N = 100

√2

For(i=2 i<=n i++){

if(isPrime(i)){

c++

}

}

N = a x b

N = a x b

a == b a > b a < b

a == b = √N b < √N a √N

For(i=2 i<=n i++){

checkPrime

}

N√

div = 2

while(div x div <= n){

if(N % div == 0){

return false

}

div++

}

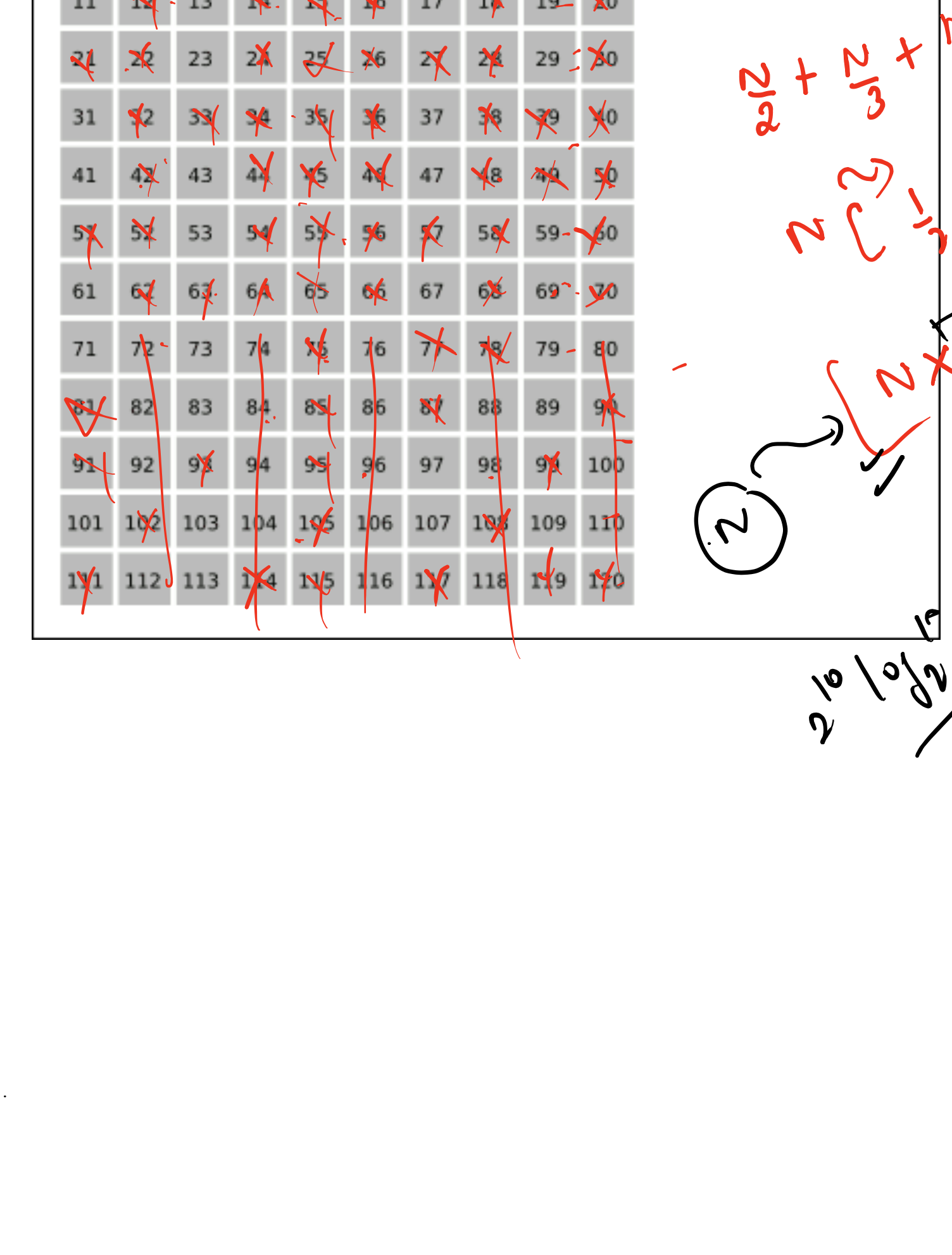
return True

div² ≤ N

div ≤ √N

i=2 i x 2 i x 3 i = 4 (j=2)

i x j ≤ n


Prime numbers

N = 120

N/2 + N/3 + N/5 + N/7 + N/11 ....

N [1/2 + 1/3 + 1/5 + 1/7 + 1/11 ....] ≈ O(n) X

N x log log N

N

N = 1024 = 2^10

2^10 x log₂ 10 log₂

2^10 x log₂ log₂ 2^10

2^10 log₂ 10 / 3

```
public static int Prime_Sieve(int n) {
        int[] prime = new int[n + 1];
        prime[0] = 1;// 0 is not a prime
        prime[1] = 1;// 1 is not a prime
        for (int i = 2; i * i < prime.length; i++) {
            if (prime[i] == 0) {
                for (int j = 2; i * j < prime.length; j++) {
                    prime[i * j] = 1;
                }
            }

        }

    }
```

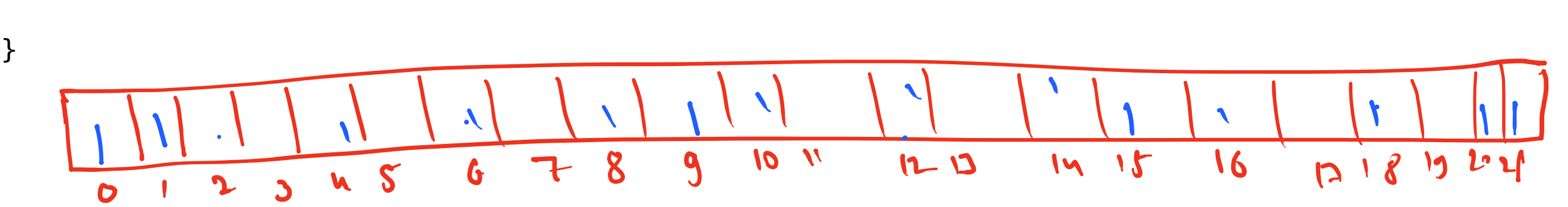
5x5 ≤ 21

3x7

i = 5

2x2 2x3 2x4 2x5 2x6 2x7 2x8 2x9 2x10

n = 21


0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21